

$$\cos \theta = \overline{a \cdot b} / \|a\| \|b\|$$

$$d(a,b) = \sqrt{\Sigma(a_i - b_i)^2}$$



FIELD GUIDE

Similarity Search

CheatSheet

— *for Retrieval-Augmented Generation*



$$J(A,B) = |A \cap B| / |A \cup B|$$

What is Similarity Search?

Similarity search is the process of finding data points that are most similar to a given query in a multi-dimensional vector space.

COMMON USE CASES

-  Document Search: Finding relevant articles, papers
-  Recommendation Systems: Products, movies, content
-  Image Recognition: Similar images, duplicates
-  Chatbots & RAG: Retrieving relevant context
-  Anomaly Detection: Unusual patterns
-  Customer Segmentation: Grouping similar users



Overview of Similarity Search Techniques

Four Core Techniques for Measuring Vector Similarity



1. Cosine Similarity

Measures angle between vectors. Best for text/document search, ignores magnitude.



2. Euclidean Distance (L2)

Straight-line distance in multi-dimensional space. Ideal for images and computer vision.



3. Manhattan Distance (L1)

Sum of absolute differences. Great for sparse data and high-dimensional spaces.



4. Dot Product Similarity

Simple element-wise multiplication and sum. Efficient for normalized embeddings.



Before moving ahead, do checkout my AI Interview Bundle

13 modules, 645 questions

[Link in caption](#)

FOUNDATIONS

Python For GenAI

55 questions

Statistics & Math for ML

30 questions

Machine Learning Fundamentals

70 questions

Deep Learning Basics

25 questions

LLM & GENAI

LLM Core Concepts

50 questions

RAG Interview Mastery

69 questions

LangChain Core Concepts

134 questions

LangGraph Core Concepts

49 questions

AGENTIC & PRODUCTION

Agentic AI

50 questions

CrewAI

53 questions

AutoGen

28 questions

FastAPI (Backend for AI)

32 questions

1. Cosine Similarity

Measures the angle between two vectors, focusing on direction rather than magnitude.

MATHEMATICAL FORMULA

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

COMPLETE IMPLEMENTATION

```
import numpy as np

def cosine_similarity(vec1, vec2):
    # Step 1: Calculate dot product (how much vectors align)
    dot_product = np.dot(vec1, vec2)

    # Step 2: Calculate magnitudes (length of each vector)
    magnitude_1 = np.linalg.norm(vec1)
    magnitude_2 = np.linalg.norm(vec2)

    # Step 3: Divide dot product by product of magnitudes
    similarity = dot_product / (magnitude_1 * magnitude_2)
    return similarity

# Example
vec1 = [1, 2, 3]
vec2 = [2, 4, 6] # Same direction, different magnitude
similarity = cosine_similarity(vec1, vec2)
print(f"Cosine similarity: {similarity}") # Output: 1.0
```



Cosine Similarity – Use Cases & Pros/Cons

BEST FOR

- ☒ Text similarity and document search
- ☒ Recommendation systems
- ☒ High-dimensional data where vector length doesn't matter
- ☒ When you care about semantic meaning, not magnitude

☒ ADVANTAGES

- Normalized (always between -1 and 1)
- Ignores document length differences
- Great for sparse vectors

☒ DISADVANTAGES

- Ignores magnitude completely
- Can be less intuitive than distance measures



2. Euclidean Distance (L2)

Calculates the straight-line distance between two points in multi-dimensional space.

MATHEMATICAL FORMULA

$$\text{euclidean_distance}(A, B) = \sqrt{(\sum (A_i - B_i)^2)}$$

COMPLETE IMPLEMENTATION

```
def euclidean_distance(vec1, vec2):  
    # Step 1: Calculate differences for each dimension  
    differences = np.array(vec1) - np.array(vec2)  
  
    # Step 2: Square all differences (removes negatives)  
    squared_differences = differences ** 2  
  
    # Step 3: Sum all squared differences and take square root  
    distance = np.sqrt(np.sum(squared_differences))  
    return distance  
  
# Convert distance to similarity  
def euclidean_similarity(vec1, vec2):  
    distance = euclidean_distance(vec1, vec2)  
    return 1 / (1 + distance)  
  
# Example  
vec1 = [1, 2, 3]  
vec2 = [2, 3, 4] # Close but not identical  
distance = euclidean_distance(vec1, vec2)  
print(f"Euclidean distance: {distance}") # Output: 1.73
```



Euclidean Distance – Use Cases & Pros/Cons

BEST FOR

-  Image similarity and computer vision
-  Data where magnitude matters (RGB values, coordinates)
-  Low to medium-dimensional data
-  When small differences in individual features are important

ADVANTAGES

- Intuitive and easy to understand
- Considers all dimensions equally
- Good for continuous numerical data

DISADVANTAGES

- Sensitive to high-dimensional data (curse of dimensionality)
- Affected by scale differences between features
- Can be dominated by features with large ranges



Manhattan Distance – Use Cases & Pros/Cons

Calculates distance by summing absolute differences along each dimension, like walking through city blocks.

MATHEMATICAL FORMULA

$$\text{manhattan_distance}(A, B) = \sum |A_i - B_i|$$

COMPLETE IMPLEMENTATION

```
def manhattan_distance(vec1, vec2):  
    # Step 1: Calculate absolute differences for each dimension  
    abs_differences = np.abs(np.array(vec1) - np.array(vec2))  
  
    # Step 2: Sum all absolute differences  
    distance = np.sum(abs_differences)  
    return distance  
  
# Example  
vec1 = [1, 2, 3]  
vec2 = [4, 1, 5]  
distance = manhattan_distance(vec1, vec2)  
print(f"Manhattan distance: {distance}")  
# Output: 6 (|1-4| + |2-1| + |3-5|)
```



4. Dot Product Similarity

Simple multiplication and summation of corresponding vector elements.

MATHEMATICAL FORMULA

$$\text{dot_product}(A, B) = \sum (A_i \times B_i)$$

COMPLETE IMPLEMENTATION

```
def dot_product_similarity(vec1, vec2):  
    # Multiply corresponding elements and sum them up  
    return np.dot(vec1, vec2)  
  
# Example  
vec1 = [1, 2, 3]  
vec2 = [2, 1, 2]  
similarity = dot_product_similarity(vec1, vec2)  
print(f"Dot product: {similarity}")  
# Output: 10 (1*2 + 2*1 + 3*2)
```



4. Dot Product Similarity

BEST FOR

- ☒ When both magnitude and direction matter
- ☒ Normalized embeddings (unit length vectors)
- ☒ Simple similarity calculations
- ☒ When you want to reward strong positive correlations

☒ ADVANTAGES

- Computationally very efficient
- Simple to understand and implement
- Good for normalized vectors

☒ DISADVANTAGES

- Sensitive to vector magnitudes
- Can produce unbounded similarity scores
- Not suitable for comparing vectors of different scales



Choosing the Right Similarity Measure

USE CASE	BEST SIMILARITY	REASON
Text/Document Search	Cosine	Focuses on semantic meaning, not document length
Image Recognition	Euclidean	Pixel differences and visual features matter
Recommendation Systems	Cosine or Dot Product	User preference patterns and correlations
Anomaly Detection	Euclidean or Manhattan	Distance from normal patterns
Sparse Data	Manhattan	Less sensitive to outliers and zeros

QUICK GUIDELINES

- 1. Text & Language → Cosine Similarity
- 2. Images & Computer Vision → Euclidean Distance
- 3. Recommendation Systems → Cosine or Dot Product
- 4. High-Dimensional Sparse Data → Manhattan Distance



GET HIRED

The only bundle you'll ever need.



Stop scattered prep. Start structured learning.

13 modules · 645 questions · built for AI engineers



Link of Bundle in caption

Want more content like this?



Tap that follow button and
stay in the loop!



Like



Comment



Share



Save